

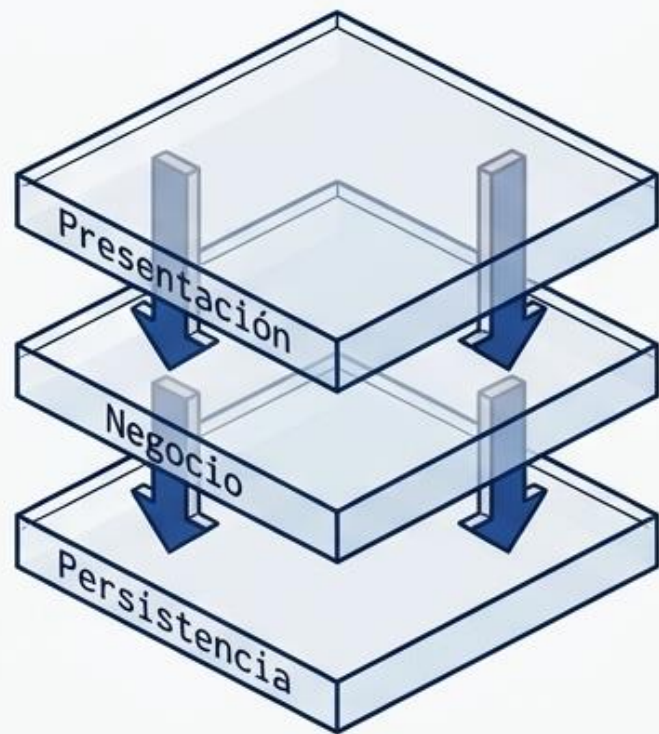
Repasemos el diseño de
patrones

El *Cheat Sheet* del 'NO': Cuándo Evitar un Patrón

Patrón	El Trade-off (Lo que sacrificas)	 Cuándo NO usarlo
Factory / Abstract	Complejidad por más clases	Si no habrá variantes o solo hay un tipo de producto.
Builder	Código más largo y verboso	Para objetos simples donde un constructor básico basta.
Singleton	Acoplamiento oculto	En sistemas concurrentes complejos (riesgo de estado global).
Adapter	Introduce una capa extra	Si tienes acceso y puedes modificar limpiamente la clase original.
Facade	Puede ocultar funcionalidad útil	Si el subsistema subyacente ya es comprensible y simple.
Decorator	Difícil de depurar visualmente	Si el comportamiento es fijo y no necesita dinamismo en runtime.
Strategy	Proliferación de clases	Si el algoritmo fundamentalmente no va a cambiar.
Observer	Riesgo de cascadas de eventos	Si el flujo de la aplicación debe ser estrictamente explícito y secuencial.

El Salto a lo Macro: Estilos Arquitectónicos Ligeros

Contexto: Dejamos los bloques de construcción individuales para hablar del plano completo del sistema. Entramos en el espíritu de Shaw & Garlan: estilos arquitectónicos, no solo código.



Arquitectura en Capas

Ventajas

Separación clara de responsabilidades. Facilita inmensamente el testeado aislado por capas.

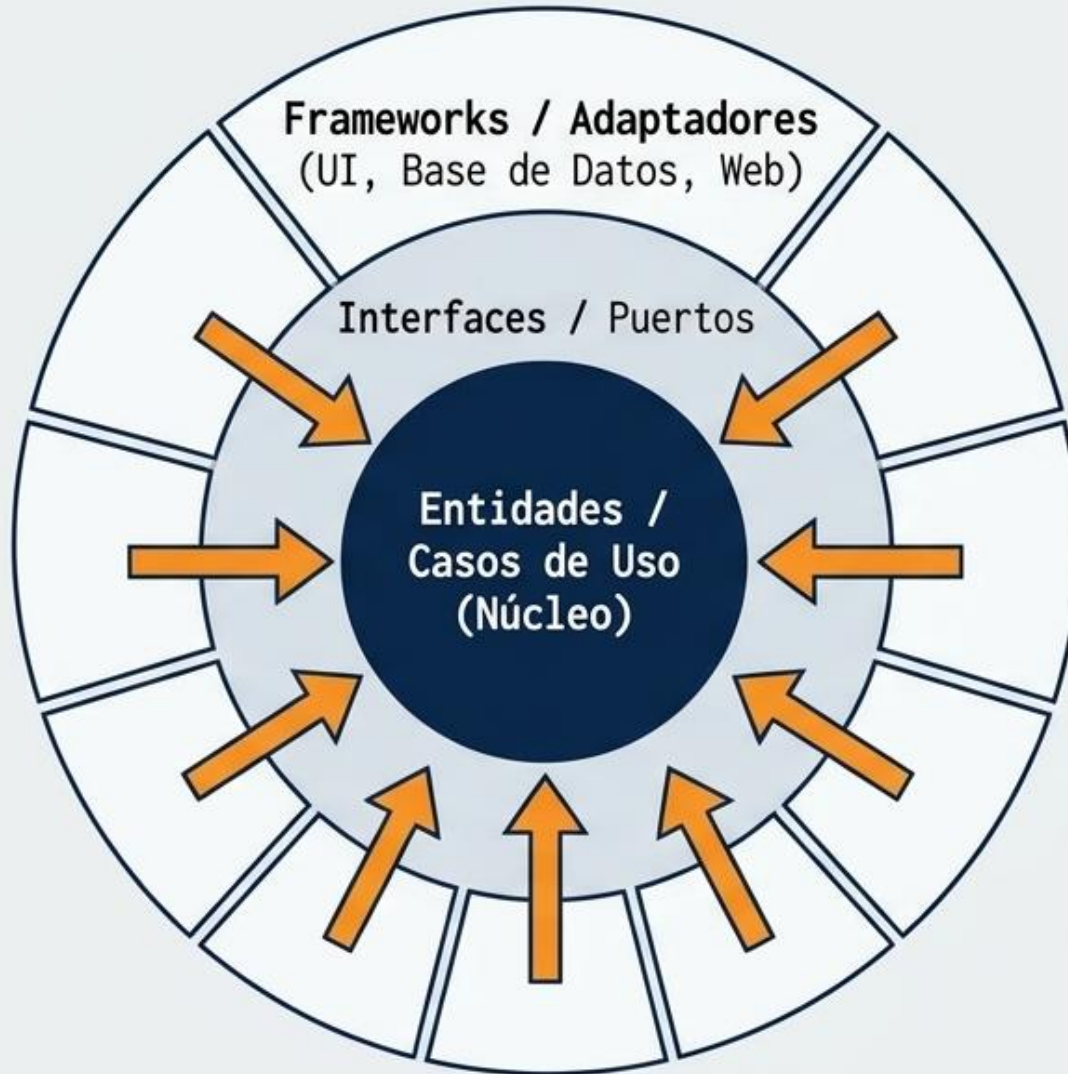
Fuerza

Modularidad vs Rendimiento (cruzar múltiples capas en cada request tiene un costo).

El Riesgo

Puede generar excesiva rigidez vertical si no se diseña con cuidado, creando "capas sumidero" que solo pasan datos sin lógica.

Arquitectura Clean y Hexagonal (Ports & Adapters)



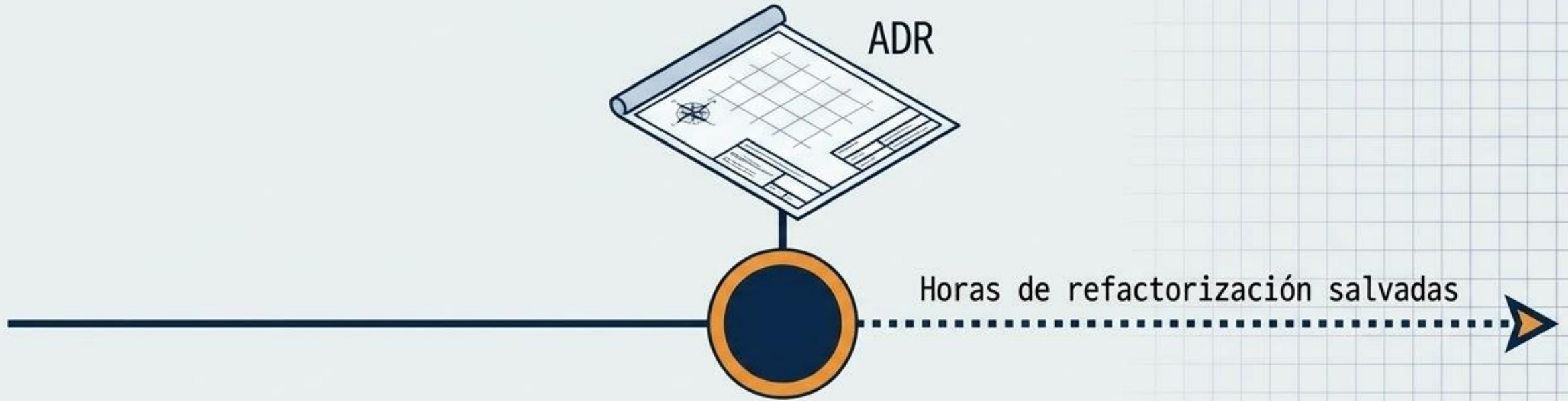
La Regla Central: Las dependencias SIEMPRE apuntan hacia el dominio.

- El sistema central (lógica de negocio) está completamente aislado.
- Los Adaptadores (UI, BD) se conectan o “enchufan” a los Puertos definidos estrictamente por el núcleo.

Fuerza

Testabilidad extrema e independencia total del framework VS. Gran complejidad conceptual y mayor carga de diseño inicial.

El Registro de Decisiones (ADR)



Architecture Decision Records

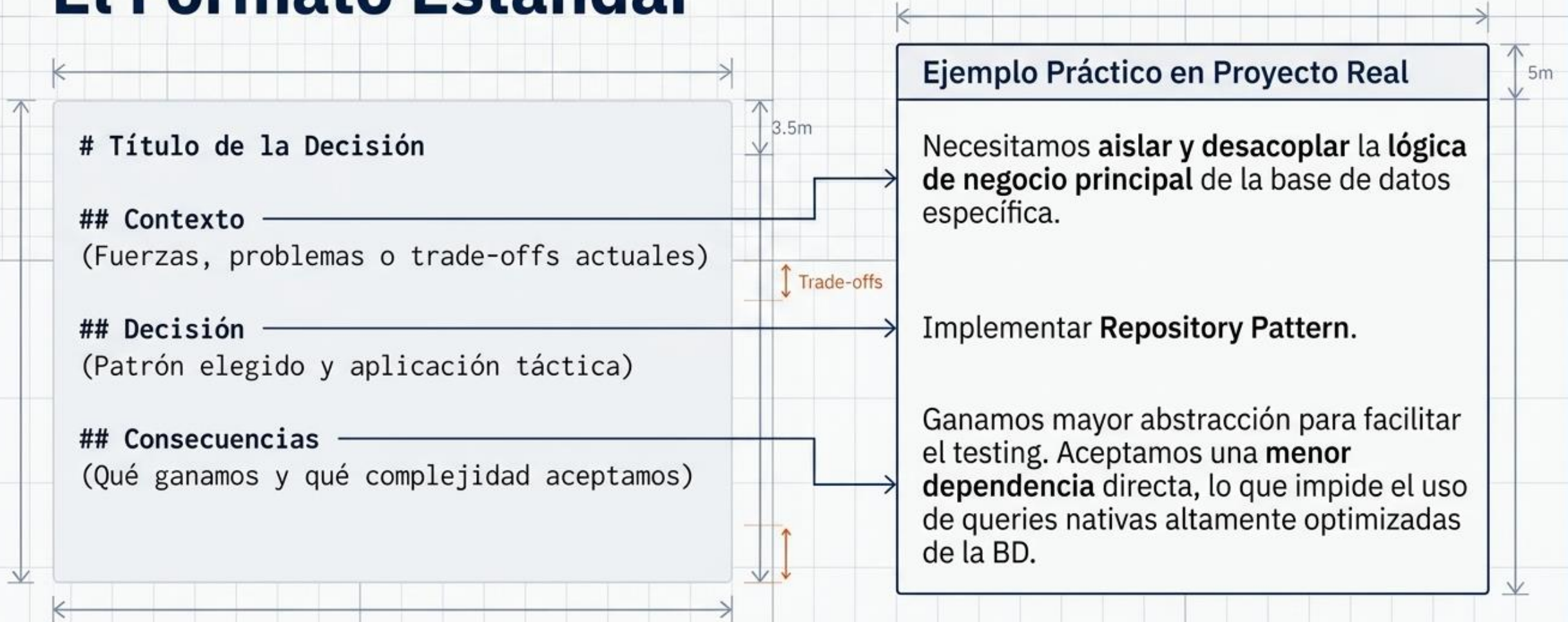
Un concepto moderno alineado a la disciplina de diseño de Shaw & Garlan. Su propósito no es documentar cómo funciona el código, sino documentar las decisiones arquitectónicas clave y, lo más importante, el contexto y las fuerzas en las que se tomaron.

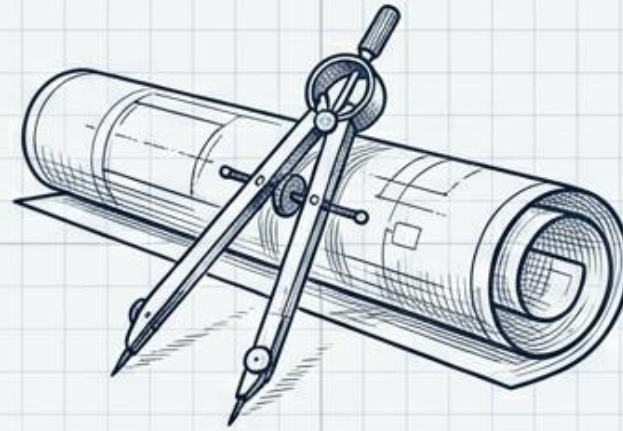
El Valor de Documentar el “Por qué”

Evita que futuros desarrolladores reviertan decisiones cruciales al no entender los *trade-offs* originales que modelaron el sistema.

Si se asume deuda técnica intencionalmente, el ADR es el comprobante.

Anatomía de un ADR: El Formato Estándar





La maestría en la arquitectura de software no es memorizar todos los patrones, sino poseer el criterio para detenerse en la solución más simple que soporte las fuerzas del sistema.

Herramientas para Profundizar

Bass, L. et al. -
Software Architecture
in Practice

Clements, P. et al. -
Documenting Software
Architectures

Buschmann, F. et al.
- Pattern-oriented
Software Architecture

Shaw, M., & Garlan, D.
- Perspectives on an
emerging discipline